ArcadeDB: One ACID Engine for Documents, Graphs, Vectors, and Time Series

[TODO: working title; alternatives in .notes strategy doc]

Taewoon Kim
HumemAI
taewoon@humem.ai

Roberto Franchini
Arcade Data
r.franchini@arcadedata.com

Luca Garulli
Arcade Data
l.garulli@arcadedata.com

***Abstract*—[TODO: Written last. Spine: multi-model unification as a real, adopted system, one page/WAL/MVCC-transaction pipeline under documents, graphs, key-value, time series, and vectors; every index type (LSM, full-text, geo, hash, dense/sparse vector) commits inside the same transaction; Raft replication ships model-agnostic WAL page diffs; one jar runs embedded in-process or as a server speaking six wire protocols. Deep dive: what adding the vector model to a unified substrate takes, including the deliberate transactional-source-of-truth / eventually-consistent-ANN-graph tradeoff. Evaluation vs specialist engines + lessons from a decade of multi-model engineering (OrientDB lineage).]**

***Index Terms*—multi-model databases, ACID transactions, vector search, graph databases, embedded databases**

I. Introduction

A production application in 2026 routinely holds three representations of the same entities: documents describing them, a graph relating them, and embeddings of them for semantic search. The standard architecture assigns each representation a specialist engine and connects them with glue code, and the glue is where correctness goes to die: there is no transaction spanning a vector store and a graph database, so every crash between writes leaves the stores disagreeing about facts they both describe. In our measurements (§VII), a composed Qdrant+Neo4j stack ended a vector-search/graph-update operation in a torn state in five of five induced failures; the same operation in one multi-model engine rolled back cleanly five of five times, and ran 6× faster end-to-end.

This paper is an industry-track systems paper about ArcadeDB, an open-source engine that takes the opposite bet: one storage substrate, one page/WAL/transaction pipeline, one replication mechanism, with documents, graphs, key-value, time series, and dense and sparse vectors as record and index types above it. The multi-model idea has an academic literature [1]–[3] and commercial claimants, but published designs unify at the API gateway over a single document core (Cosmos DB [4]), or route to multiple specialized engines under one service (X-Stor [5], M2 [6]). To our knowledge no native multi-model *operational* engine that unifies all models at the storage layer has a systems paper at a major database venue; the closest is the workshop paper on OrientDB [7], ArcadeDB's direct ancestor. We claim the unification, not the parts: every individual mechanism (paged storage, LSM indexing, MVCC-style optimistic transactions, Raft) is deliberately conventional; what is uncommon is that one instance of each serves five data models, and what that buys and costs in practice.

Concretely, we contribute: (1) the architecture of a storage-level multi-model engine, with the precise inheritance rule that makes model count cheap (a model whose writes are paginated gets durability, transactions, and replication by construction, §III); (2) a case study of adding the vector model to that substrate, including the one deliberate relaxation, an eventually-consistent derived ANN graph over transactional vector records, and what integration cost in measured memory and latency (§IV); (3) an evaluation against eight specialist and embedded engines on community benchmarks (LDBC, TPC-H, Big-ANN, DEEP, TSBS) with exact ground truth, disclosed durability contracts, and $N{=}5$ dispersion on every number, exceeding the statistical-rigor norm of this track; and (4) engineering lessons with production evidence, including seven engine defects found by this paper's own harness and fixed upstream during its writing (§VIII).

We write in an honest register throughout: ArcadeDB loses columnar analytics by three orders of magnitude, loses bulk time-series ingest, builds ANN indexes slower than specialists, and one of our own findings is a sparse retrieval latency cliff on real term distributions that specialist engines do not exhibit. The paper's argument is not that one engine beats N specialists at their own games; it is that for operational workloads spanning models, storage-level unification delivers competitive per-model performance plus cross-model transactions that composed stacks cannot offer at any price.[TODO: deployment/adoption sentence with metrics.arcadedb.com numbers + Lucas Health + DatAasee; running-example figure F2 reference]

II. Background and Design Goals

[TODO: PROSE PASS NEEDED. Raw co-author input below (Garulli, 2026-07-08).]

ArcadeDB descends from OrientDB, and its design goals are best understood as deliberate corrections of three OrientDB limitations its authors lived with:

- **Durability.** OrientDB's write-ahead log was asynchronous, single-threaded, and (in the authors' assess-

ment) unreliable. ArcadeDB's journal is synchronous, parallel, and crash-proof. It follows the point-of-no-return discipline detailed in §III.

- **Edge storage.** OrientDB managed edges with a tree-bag structure; ArcadeDB replaces it with a LIFO linked list that scales substantially better at high edge counts.
- **High availability.** The OrientDB multi-master model "simply doesn't work" (authors' words); ArcadeDB abandons it for a Raft leader/follower architecture (§VI).

[TODO: Design goals framing; what industry usage demanded. Third-party academic adoption: DatAasee [8] (metadata catalog for research libraries) and AptBacterialDB [9] (ArcadeDB/openCypher graph layer for an antibacterial-aptamer database), both independent multi-model graph uses. Christian's "why we chose it" paragraph goes here or in Lessons.]

III. Architecture: One Substrate, Five Models

ArcadeDB is organized around a single storage abstraction rather than a federation of per-model engines. Everything that persists, whether it holds documents, graph adjacency, key-value pairs, time-series points, or vector index segments, is a *paginated component*: a file of fixed-size pages managed by one page manager, modified through one transaction context, and made durable by one write-ahead log. The data models differ in the record types and index structures layered on top; they do not differ in how bytes reach disk, how changes become durable, or how replicas stay consistent. This section walks that pipeline bottom-up and states precisely which guarantees each model inherits from it.

A. Pages, Transactions, and the Log

All reads and writes go through page-level copy-on-write. A transaction that modifies any record obtains private copies of the affected pages from the page manager and accumulates its changes there; concurrent readers continue to see the unmodified originals. Commit is optimistic: at first phase, the engine validates that the pages read by the transaction have not been changed by a concurrent committer (a per-page version check), then serializes every dirty page, *including index pages*, into a single WAL entry. There is no separate index log and no index-specific recovery path: a secondary-index update, a graph edge insertion, and a document field change that happen in the same transaction land in the same log record and either all survive a crash or none do. Recovery replays the WAL forward from the last checkpoint; we measure recovery time and post-crash integrity under `kill -9` in §VII.

Durability is configurable per deployment in one dimension only: when the WAL buffer reaches the operating system (asynchronously on a background flusher by default, or `fsync` per commit under the strictest setting). We flag this knob here because it moves throughput by more than an order of magnitude and is therefore the first thing our evaluation pins when comparing against engines with different default contracts (§VII).

TABLE I
Storage and index structures. Every row is a paginated component: same pages, same WAL entry at commit, same replication path. The ANN graph (marked †) is derived state, rebuilt rather than replicated (§IV).

Structure	Serves	Implementation
Buckets	documents, vertices, edges, KV	paged record file
LSM tree	secondary indexes (unique/non)	mutable + compa
Full-text	text search	analyzer over LS
Geospatial	spatial predicates	curve keys over
Adjacency	graph traversal	RID lists, light e
`LSM_SPARSE_VECTOR`	learned sparse retrieval	postings as LSM
Vector segments	dense ANN	records + JVecto
Time-series buckets	append/range by time	paged, SIMD co

B. One Index Family Tree

Table I inventories the index and storage structures. The workhorse is an LSM tree [10] implemented as a paginated component: mutable in-memory pages absorb writes, immutable compacted segments serve reads, and compaction is an ordinary paginated write path. The full-text index is a thin analyzer layer over the same LSM tree (tokens as keys), and the geospatial index likewise reduces to LSM entries over space-filling-curve keys. The practical consequence of this reuse is uniform transactional behavior: because every index is a paginated component, the commit path of §III-A covers all of them with no per-index machinery. The two vector index families are the newest members of the tree and the most demanding test of the claim that the substrate generalizes; they get their own section (§IV).

C. Graph Storage, Stated Honestly

Vertices and edges are records in buckets, and adjacency is maintained as RID-linked lists attached to each vertex, a design inherited from OrientDB [7] and refined (the tree-bag edge container was replaced by a LIFO linked list that behaves better at high fan-out, §II). Edges without properties are stored as *light edges*: pure RID pairs in the adjacency structure with no edge record at all, which roughly halves the storage and I/O cost of plain connections. Traversal is pointer-chasing over the record store through the same page cache as every other access. We do not claim parity with purpose-built native graph storage on every workload; we claim, and measure in §VII, that RID-linked adjacency over a shared substrate delivers competitive point-traversal latency while inheriting transactions, recovery, and replication that a bolted-on graph layer would have to re-implement.

D. Replication Is Model-Agnostic

Because every model's changes reduce to WAL'd page deltas, replication does not know what a document, an edge, or a vector is. The high-availability module (§VI) ships transaction page deltas through a Raft log [11]; followers apply them through the same recovery code path used after a crash. Adding a new data model to ArcadeDB, as §IV recounts for vectors, required *no* replication work: a model whose writes

are paginated is replicated by construction. The one deliberate exception, derived search structures that are rebuilt rather than replicated, is discussed in §IV.

IV. Case Study: Adding the Vector Model

Vector search is the sharpest recent test of the unification thesis: it arrived as a workload after the substrate was fixed, it is dominated by specialist engines with no transactional obligations, and it stresses exactly the properties (index build cost, memory, approximate results) that a shared substrate might be suspected of handling poorly. This section describes what adding dense and sparse vector search to ArcadeDB actually took, in the spirit of integration case studies such as SingleStore-V [12] and Cosmos DB's DiskANN integration [13], and states the one place where we deliberately relaxed the substrate's guarantees.

A. Dense Vectors: Transactional Records, Derived Graph

Dense embeddings are ordinary record properties: they live in buckets, are WAL'd, versioned, and replicated like any other field. Approximate search is served by a graph index built with the JVector library [14], which combines an HNSW-style layer hierarchy [15] with Vamana-style graphs within each layer [16]. Here we made the one deliberate exception to full unification: the ANN graph itself is an asynchronously maintained, WAL-disabled structure. Vector *records* are the transactional source of truth; the search graph is derived state, rebuilt from records if lost, and followers rebuild rather than receive it. The alternative, WAL-logging every neighbor-list mutation of an incremental graph build, would have coupled commit latency of unrelated transactions to ANN maintenance. The consequence is a bounded staleness window between a committed insert and its visibility in approximate search (newly inserted vectors are meanwhile served exactly from a delta buffer), which we consider the correct trade for an operational engine and state openly rather than claim a fully transactional ANN index. One design decision this enabled: because vectors-as-records are the ground truth, index parameters can be changed and the graph rebuilt without touching data, which we used in §VII to match graph degrees fairly across engines.

B. Sparse Vectors: An Inverted Index That Commits

Learned sparse retrieval (SPLADE-style [17]) maps text to high-dimensional sparse weight vectors and is conventionally served by standalone inverted-index engines with block-max pruning [18]–[20]. To our knowledge ArcadeDB is the first general-purpose transactional DBMS to ship this workload natively: `LSM_SPARSE_VECTOR` stores posting lists as LSM segments (the same paginated component family as every other index), with RID-delta compression, per-block weight quantization to int8, block maxima for WAND-style pruning at query time, and size-tiered compaction as the settle step. Because postings ride the LSM commit path, sparse index updates are transactional with document writes, a property none of the standalone engines offer. The quantization choice is a measured trade: int8 posting weights cost about half a point of recall@10 against exact scoring (§VII) and can be disabled per index (fp32 segments) where exact weights matter more than space.

C. What Integration Cost, Measured

Honesty about the bill: the substrate gave vectors durability, transactions, and replication for free, but not specialist performance for free. Three costs surfaced in our own measurements (§VII, §VIII): graph index construction at 10M vectors requires several times the raw vector size in heap because build-time structures hold vectors on-heap alongside JVector's construction state; sparse query latency on real SPLADE term distributions degrades at corpus sizes where specialist engines do not; and settle steps (compaction) that specialists hide behind background services are visible operations here. We report these as findings with the same prominence as the wins, and several became upstream fixes during the course of this work (§VIII).

D. A Note on Measurement Integrity

The sparse index's original design documentation reported a 766× speedup over brute-force scoring at 1M documents, a number that survived review because the same document also *retracted* an earlier 41,613× claim traced to a broken baseline. We mention this not as trivia but as method: every headline number in this paper carries its measurement protocol, and where our own instruments were later found misleading (a memory metric that counted reclaimable page cache; a timeout that was our harness's fault) the corrected number replaced the flattering one. §VII states the protocol once, and the raw results are published with the paper.

V. Query and Deployment Surfaces

Unification below enables plurality above. One query executor and one result model sit behind four query languages (SQL with graph-traversal extensions, Cypher, GraphQL, and the MongoDB query language; Gremlin runs via TinkerPop over the same storage), so a Cypher `MATCH`, a SQL join to the same vertices, and a Mongo-style document filter compile to plans over identical components and compose in one transaction. Language support is a translation concern, not a storage concern, which is what keeps four surfaces maintainable by a small team.

The same engine deploys two ways from one artifact. Embedded, it runs in-process (JVM native; Python via the bindings whose internals are described elsewhere[TODO: cite SciPy paper when published]) with function-call latency and no server to operate, the mode SQLite [21] and DuckDB [22] made standard for their respective niches but that no multi-model or vector-capable engine occupied. As a server, the identical engine speaks HTTP/JSON, the PostgreSQL wire protocol, Redis, Mongo, and Bolt, letting existing drivers connect unmodified. The embedded and server rows in every evaluation table are the same engine and storage format; §VII quantifies the transport fee, and teams in practice prototype embedded and promote to server without a migration.

TABLE II
TABULAR WORKLOADS, MEDIUM SCALE + TPC-H SF1. MEDIAN [MIN–MAX], N=5. DURABILITY CONTRACTS DIFFER: ARCADEDB GROUPS COMMITS (ASYNC WAL FLUSH, DEFAULT); POSTGRESQL AND DUCKDB FSYNC PER COMMIT. MATCHED-CONTRACT PAIRING IN THE TEXT.

System	Ingest (rows/s)	OLTP (ops/s)	Insert p99 (ms)	TPC-H Q1 (ms)
ArcadeDB (emb)	31,282 [31,104–31,558]	5929 [5450–6058]	0.540 [0.518–0.574]	96 [90–102]
ArcadeDB (srv)	27,421 [27,003–27,565]	1288 [1204–1337]	1.45 [1.30–1.50]	10,693 [10,520–11,002]
DuckDB	323,005 [319,708–329,174]	268 [266–269]	10.02 [9.93–10.18]	15.54 [15.43–16.08]
PostgreSQL	331,924 [328,504–336,735]	525 [514–531]	2.94 [2.89–3.04]	990 [986–999]

TABLE III
LDBC SNB GRAPH WORKLOADS, SF1/SF10. MEDIAN [MIN–MAX], N=5. BIDIRECTIONAL EDGE POINTERS (ENGINE DEFAULT) FOR ALL ARCADEDB ROWS.

System	Scale	Build (s)	1-hop p50 (ms)	1-hop p99 (ms)	2-hop p99
ArcadeDB (emb)	SF1	2.30 [2.24–2.36]	0.488 [0.428–0.549]	1.47 [1.34–1.93]	6.91 [6.64–
ArcadeDB (emb)	SF10	26.23 [25.84–28.68]	0.559 [0.555–0.688]	1.38 [1.37–1.93]	18.23 [15.48–2
ArcadeDB (srv)	SF1	8.56 [8.50–8.81]	1.25 [1.19–1.34]	2.42 [2.34–2.50]	9.28 [8.93–
ArcadeDB (srv)	SF10	74.25 [73.53–84.07]	1.37 [1.22–1.52]	2.71 [2.23–2.92]	22.45 [18.16–2
Neo4j	SF1	7.37 [7.15–7.69]	4.92 [4.82–4.96]	7.51 [7.01–8.05]	7.75 [7.54–
Neo4j	SF10	45.74 [45.25–46.47]	5.30 [5.27–5.43]	6.86 [6.61–7.55]	10.50 [9.96–
LadybugDB	SF1	0.520 [0.470–0.530]	1.12 [1.07–1.13]	1.41 [1.35–1.54]	5.25 [4.57–
LadybugDB	SF10	3.52 [3.47–3.55]	1.66 [1.55–1.89]	4.49 [3.66–5.39]	11.04 [10.90–

TABLE IV
SPARSE RETRIEVAL ON BIG-ANN (REAL SPLADE/MS MARCO), EXACT MIPS GROUND TRUTH, 1000 DEV QUERIES. MEDIAN [MIN–MAX], N=5. INT8 ROWS USE THE ENGINE'S DEFAULT POSTING QUANTIZATION; FP32 DISABLES IT.

System	Corpus	Build (s)	p50 (ms)	p99 (ms)	
ArcadeDB (emb, int8)	100k	5.70 [5.63–5.73]	1.45 [1.41–1.57]	3.22 [3.17–3.33]	0.9
ArcadeDB (emb, int8)	1M	154 [153–155]	165 [161–167]	581 [567–593]	0.9
ArcadeDB (emb, fp32)	100k	5.63 [5.60–5.67]	1.50 [1.45–1.68]	3.39 [2.85–3.56]	
ArcadeDB (emb, fp32)	1M	153 [152–155]	164 [155–167]	575 [520–603]	
ArcadeDB (srv)	100k	10.25 [10.20–10.37]	2.76 [2.59–3.01]	4.29 [4.14–5.76]	0.9
ArcadeDB (srv)	1M	322 [318–325]	194 [194–196]	699 [696–712]	0.9
Qdrant	100k	2.92 [2.90–2.94]	0.682 [0.671–0.768]	2.40 [0.922–2.50]	
Qdrant	1M	96.59 [90.82–97.26]	2.91 [2.76–2.92]	4.11 [4.06–4.28]	
Milvus	100k	5.36 [5.31–5.42]	0.807 [0.800–0.820]	1.20 [0.963–1.36]	
Milvus	1M	98.63 [98.26–102]	9.29 [8.59–9.33]	32.51 [30.12–34.27]	
Elasticsearch	100k	17.35 [17.28–17.38]	2.33 [2.32–2.41]	5.47 [4.61–6.24]	0.9
Elasticsearch	1M	325 [324–328]	3.58 [3.32–3.69]	7.57 [7.05–8.14]	0.7

VI. HIGH AVAILABILITY

The HA module replicates at the same layer everything else lives: committed transactions ship as WAL page deltas through a Raft log [11] (Apache Ratis), and followers apply them with the crash-recovery code path. Leader election, membership, and quorum are Ratis's; ArcadeDB's contribution is the state machine, which is small precisely because deltas are model-agnostic bytes. The design was adopted after the predecessor system's multi-master replication proved operationally unsound (§II); the trade accepted is a single writer per database. Under write load, killing the leader yields resumed writes in 0.2–3.3 s with no acknowledged-write loss in our trials (§VII). Because replication never inspects model types, the vector, graph, and time-series models documented in this paper were replicated correctly the day they were merged, with one deliberate carve-out: derived ANN graphs are rebuilt on followers rather than shipped (§IV).

VII. EVALUATION

The evaluation asks one question per data model (is ArcadeDB competitive with the specialist a practitioner would otherwise deploy?) and one question no specialist can answer (what does one engine buy when the workload spans models?). We report losses with the same prominence as wins.

Protocol. All measurements run on a dedicated Intel i9-12900HK (6 P-cores/20 threads used as pinned cpusets, 61 GiB RAM, NVMe SSD, Linux 7.0), every system in Docker with pinned image digests, identical CPU and memory caps per cell, and JVM heap fixed (`-Xms=-Xmx`) for latency stability. Each cell is N=5 independent repetitions; tables report median [min–max]. Quality metrics (recall@10 against exact ground truth) are reported next to every approximate-search latency. Durability contracts differ across engines by default and move results by orders of magnitude; we state each engine's contract per table and additionally measure matched-contract pairs. Raw results, harness, and per-cell manifests are published.[TODO: artifact URL; October freeze re-measure on one pinned release; comparator version list]

A. Tabular: Documents as Rows

Table II (medium scale; TPC-H SF1 for the analytical queries). ArcadeDB embedded sustains 5,929 OLTP ops/s with 0.54 ms insert p99, ahead of PostgreSQL (525 ops/s) and DuckDB (268 ops/s), but the contracts differ: ArcadeDB groups commits through an asynchronous WAL flusher by default while PostgreSQL and DuckDB fsync per commit; under ArcadeDB's strictest per-commit fsync the gap closes to rough parity with PostgreSQL. The analytical side is an honest loss: link-based row storage answers TPC-H Q1 in 10.7 s where columnar DuckDB needs 15.5 ms. ArcadeDB is an operational engine; workloads dominated by full-column scans belong on a columnar system, and §VIII discusses where that boundary sits.

B. Graph: LDBC Social Network

Table III (LDBC SNB persons/knows, SF1 and SF10). Point traversals are ArcadeDB's strongest surface: 1-hop p50 of 0.49 ms embedded (0.56 ms at SF10) versus 4.9 ms for Neo4j and 1.1 ms for LadybugDB, with p99 under 1.5 ms at both scales. RID-linked adjacency holds its shape as the graph grows tenfold. The loss column: SF10 2-hop tails favor Neo4j (10.5 ms p99 vs. our 18.2 ms), consistent with a planner that has decades of traversal-ordering work behind it.

C. Dense Vectors: DEEP-10M

Table V top half (10M × 96d, exact GT, graph degrees matched across engines). ArcadeDB embedded reaches 0.950 recall@10 at 5.5 ms p50: mid-pack among eight systems, behind Qdrant (0.980 at 1.3 ms) and Chroma, ahead of exact scanners and of LanceDB's operating point (0.707 recall). Index build is the weak axis (2,796 s vs. Qdrant's 516 s) and its memory profile is dissected in §VIII. We consider mid-pack

TABLE V
DENSE ANN ON DEEP-10M (DEGREE-MATCHED, EXACT GT) AND TSBS CPU-ONLY TIME SERIES. MEDIAN [MIN–MAX], N=5.

Dense ANN, DEEP-10M (10M×96d), degree-matched			
System	Build (s)	p50 (ms)	p99 (m
ArcadeDB (emb)	2796 [2772–2840]	5.45 [5.42–5.53]	17.92 [14.47–50.5
ArcadeDB (srv)	3709 [3701–3736]	6.82 [6.59–7.10]	107 [105–10
Qdrant	516 [513–524]	1.29 [1.17–1.38]	1.73 [1.71–1.9
Milvus	142 [137–146]	17.94 [13.01–18.84]	24.30 [22.59–26.7
Chroma	3679 [3673–3696]	0.686 [0.632–0.726]	0.896 [0.797–0.96
LanceDB	227 [226–228]	2.72 [2.64–2.75]	441 [368–56
sqlite-vec	70.26 [69.95–70.75]	882 [879–884]	900 [897–90
DuckDB-VSS	805 [805–806]	2.53 [2.52–2.54]	5.56 [5.27–6.4

Time series, TSBS cpu-only (2.59M points)				
System	Ingest (pts/s)	Last point (ms)	1h bucket (ms)	12h global (ms)
ArcadeDB (emb)	31,276 [30,957–31,509]	0.520 [0.490–0.580]	4.80 [3.92–5.28]	1.92 [1.82–1.81]
DuckDB	1,940,570 [1,910,007–1,946,089]	1.40 [1.37–1.54]	1.28 [1.24–1.84]	4.5 [4.07–4.82]
QuestDB	431,306 [428,659–436,302]	0.850 [0.720–0.890]	0.770 [0.720–0.85]	2.22 [2.05–2.82]

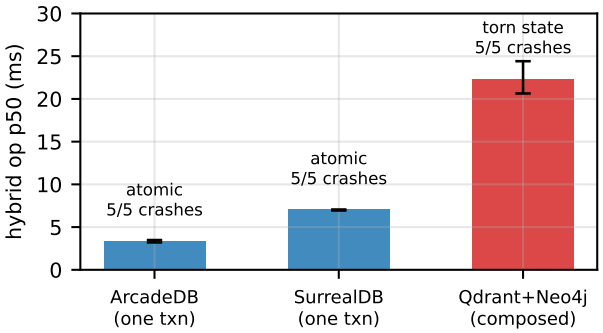

Fig. 2. Cross-model operation (vector hit → graph expansion → document update): end-to-end p50 and the outcome of killing the writer mid-operation, 5 trials each. The composed stack has no transaction boundary spanning its stores.

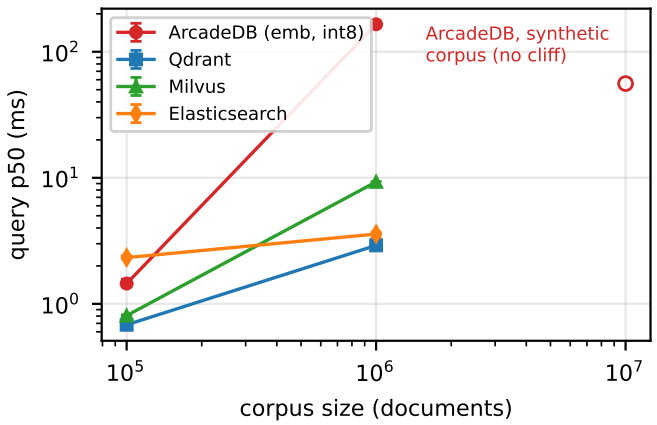

Fig. 1. Sparse retrieval p50 vs. corpus size on real SPLADE vectors (Big-ANN), log-log, median [min–max] over N=5. ArcadeDB is competitive at 100k and cliffs at 1M; the same engine on a synthetic corpus shows no cliff even at 10M (hollow marker), locating the problem in real term distributions (§VIII).

latency at matched recall, plus transactions and replication the specialists lack, the honest summary of where integration stands today.

D. Sparse Vectors: Big-ANN SPLADE

Table IV and Fig. 1 (real SPLADE/MS MARCO vectors, 100k and 1M tiers, exact MIPS GT). At 100k ArcadeDB is competitive (1.45 ms p50, recall 0.996, twice Qdrant's latency). At 1M it is not: p50 degrades to 165 ms, a cliff our specialists do not exhibit, which we traced to head-term posting lists on real term distributions (§VIII; upstream issue filed during this work). The synthetic corpus used by the engine's own benchmarks shows no cliff at 10M, which is itself a finding about synthetic sparse benchmarks. Elasticsearch offers a mirror-image trade at 1M: fast (3.6 ms) but at 0.756 recall from aggressive default pruning, versus ArcadeDB's 0.994.[TODO: update after upstream fix + 8.84M tier at freeze]

E. Time Series: TSBS

Table V bottom half. Specialists win ingest by 14–62× (DuckDB 1.94M pts/s, QuestDB 431k, ArcadeDB 31k) and ArcadeDB loses global aggregation outright (1.8 s vs. milliseconds; no time partitioning). It wins the operational lookup: last-point-per-host at 0.52 ms p50 beats both specialists through the composite (host, ts) index. The lane's purpose is scope honesty: the substrate handles the shape, and applications with an embedded time-series side channel get it for free, but a dedicated TSDB it is not.

F. Cross-Model: One Transaction vs. a Composed Stack

The workload no specialist runs (Fig. 2): a vector hit expands over graph edges and updates a document, atomically. In ArcadeDB this is one transaction at 3.36 ms median end-to-end [3.20–3.49]; SurrealDB (the one embeddable multi-model rival) needs 7.02 ms; a composed Qdrant+Neo4j stack needs 22.35 ms of network and glue. Then the failure injection: killing the writer mid-operation left the composed stack in a torn state in 5 of 5 trials (vector store and graph disagree on the result), while ArcadeDB and SurrealDB rolled back cleanly in 5 of 5. This is the paper's thesis in one experiment: the composed stack's inconsistency is not an implementation bug but the absence of a transaction boundary spanning the models.

G. Durability and Failover

Under `kill -9` during sustained ingest (600k+ rows in flight), ArcadeDB recovers in under one second with zero torn records across five trials (watermark-verified), both under the default and the strict WAL contract. A three-node Raft cluster under write load elects and resumes writes 0.2–3.3 s after the leader is killed, with all acknowledged writes present afterward in 5 of 5 trials. Replication required no vector- or graph-specific code (§III-D).

H. Embedded vs. Server

Across lanes, the same engine in-process versus behind HTTP: OLTP 5,929 vs. 1,288 ops/s, graph 1-hop 0.49 vs. 1.25 ms, dense p50 5.45 vs. 6.82 ms. The deployment axis

costs a fixed per-operation transport fee that dominates point workloads and vanishes for scan-bound ones; one engine spans both deployment points, which neither SQLite-class embedded engines nor server-only specialists offer.[TODO: F8 figure]

VIII. Lessons Learned and Tradeoffs

Benchmarks compare durability contracts before they compare engines. Our first tabular results showed ArcadeDB an order of magnitude ahead of engines that were simply fsyncing more honestly. Every cross-engine throughput table in the literature that omits the commit contract should be read with suspicion; ours state it per table, and the strict-contract pairing moved one headline result by two orders of magnitude.

Synthetic benchmark data hides real behavior. The sparse index shows no latency cliff at 10M synthetic documents and a 100× cliff at 1M real SPLADE documents: real term distributions concentrate mass in head terms whose posting lists dominate scoring. A per-query analysis over the 1000 dev queries makes it precise: latency is rank-correlated 0.95 with the summed posting length of a query's terms, the signature of an exhaustive document-at-a-time pass, because SPLADE's query expansion gives every query several head terms and flat weights that defeat the engine's block-max pruning. The engine's own regression benchmarks, built on synthetic corpora, could not have caught this. Community datasets with exact ground truth (Big-ANN, DEEP, LDBC, TPC) are not academic formality; they are where the bugs are.[TODO: fix status of upstream issue #5388 at freeze]

Matched operating points, not matched parameter names. JVector's `maxConnections` is a per-layer degree where hnswlib's M doubles at the base layer, so equal parameter values silently compare graphs of half the degree. Discovering this raised ArcadeDB's dense recall from 0.87 to 0.95 at matched cost and became an upstream default change. Fair comparison requires matching what parameters *do*, not what they are called.

The build-side bill of on-heap integration. Dense index construction at 10M vectors needs roughly 5× the raw vector size in JVM heap: the engine holds vectors in a delta buffer and, during build, a second boxed copy in the builder's cache, alongside JVector's graph construction state. We documented the breakdown upstream during this work; bounding either copy is mechanical, and a PQ-scored build path already in the code would cut it further. The lesson generalizes: integrating a library that assumes ownership of memory into an engine that also does requires an explicit budget, or the sum exceeds both.

Working in the open converts benchmarking into engineering. Seven engine issues found by this paper's harness were filed with reproductions and fixed upstream, most within days, several changing engine defaults. A benchmark against a closed system produces a table; against an open system with a responsive maintainer it produces a better system and then a table. The October re-measurement on a single pinned release will carry those fixes.

Where the specialists keep winning. Columnar scans (three orders of magnitude on TPC-H Q1), bulk time-series ingest (62×), ANN build time (5×), and the last factor of 2–4 in specialist query latency at matched recall. We consider none of these disqualifying for an operational multi-model engine, and all of them worth stating plainly. [TODO: production war stories (Roberto); adoption-paradox paragraph; metrics.arcadedb.com numbers here or in intro]

IX. Related Work

Multi-model systems. Lu and Holubová's survey [1] distinguishes single-engine multi-model systems from polyglot federations; benchmarks followed (UniBench [2], M2Bench [3]). Among engines, Cosmos DB's published design projects every API (SQL, Mongo, Cassandra, Gremlin) onto one schema-agnostic document core [4]: multi-model at the API layer over a single storage model, with graph edges stored as documents. Recent cloud services go the other way and route models to multiple specialized engines (X-Stor [5]; the M2 analytic system [6]). ArangoDB and SurrealDB, the commercial systems closest in spirit, have no peer-reviewed engine papers; OrientDB has one workshop paper [7]. ArcadeDB occupies the unclaimed cell: one storage engine, model-specific record and index types, unification at the page/WAL/transaction/replication layer, described at a database venue.

Vectors in databases. Purpose-built vector DBMSs established the workload [23], [24]; a counter-thread integrates vector search into general-purpose engines: AnalyticDB-V [25], PASE [26], VBASE [27], SingleStore-V [12], and Cosmos DB's DiskANN integration [13], which argues explicitly that vector indices belong inside operational databases. Our case study extends that argument to an embeddable multi-model engine and adds the learned-sparse workload, which prior integrations do not cover. Index algorithms are imported, not invented, here: HNSW [15] and Vamana [16] via JVector [14].

Sparse retrieval. Learned sparse models (SPLADE [17]) are served by inverted indexes with dynamic pruning, from WAND [18] through Block-Max WAND [19] to recent learned-sparse-specific structures [28]; Tonellotto et al. survey the area [20]. This literature lives on standalone index engines; to our knowledge ArcadeDB is the first transactional DBMS shipping block-max sparse retrieval natively, and our 1M finding (§VII) shows real term distributions stress exactly the part this literature optimizes.

Embedded engines. SQLite [21] and DuckDB [22], [29] define the embedded poles (operational and analytical) and Berkeley DB the lineage [30]; none spans embedded-to-replicated-server with one artifact, the continuum §V describes.

Industry-track systems papers. Our structure follows the unification narratives of Procella [31] and Snowflake [32] and the integration case-study form of SingleStore-V [12], applied to the data-model axis rather than the serving/analytics axis.

X. Conclusion

ArcadeDB demonstrates that storage-level multi-model unification is buildable and operable: one page/WAL/transaction/replication pipeline serving five data models, competitive with specialists on operational workloads, honest about where it is not, and uniquely able to give cross-model operations a transaction boundary. The measured bill of unification is real but itemizable, and several line items shrank upstream during the writing of this paper. We offer the design, the numbers, and the lessons as a reference point for the next engine that considers taking the same bet.

Acknowledgment

We thank Christian Himpe (University of Münster) for his contributions to ArcadeDB, for sharing his experience operating it in DatAasee [8], and for feedback on this manuscript.

AI-generated content disclosure (per ICDE policy): large language models (Anthropic Claude) were used as drafting and editing assistants for prose in all sections and for benchmark-harness code under author direction; all measurements, claims, and conclusions were verified by the authors, who take full responsibility for the content.[TODO: keep current at submission]

References

[1] J. Lu and I. Holubová, "Multi-model databases: A new journey to handle the variety of data," *ACM Computing Surveys*, vol. 52, no. 3, pp. 55:1–55:38, 2019.

[2] C. Zhang, J. Lu, P. Xu, and Y. Chen, "UniBench: A benchmark for multi-model database management systems," in *TPC Technology Conference (TPCTC)*, ser. LNCS 11135, 2019, pp. 7–23.

[3] B. Kim, K. Koo, U. Enkhbat, S. Kim, J. Kim, and B. Moon, "M2Bench: A database benchmark for multi-model analytic workloads," *Proceedings of the VLDB Endowment*, vol. 16, no. 4, pp. 747–759, 2022.

[4] D. Shukla, S. Thota, K. Raman, M. Gajendran, A. Shah, S. Ziuzin, K. Sundaram, M. G. Sowell, J. Levandoski, D. Lomet *et al.*, "Schema-agnostic indexing with Azure DocumentDB," *Proceedings of the VLDB Endowment*, vol. 8, no. 12, pp. 1668–1679, 2015.

[5] H. Lei *et al.*, "X-Stor: A cloud-native NoSQL database service with multi-model support," *Proceedings of the VLDB Endowment*, vol. 17, no. 12, pp. 4025–4037, 2024.

[6] B. Kim *et al.*, "M2: An analytic system with specialized storage engines for multi-model workloads," arXiv:2508.02508, 2025.

[7] D. Ritter, L. Dell'Aquila, A. Lomakin, and E. Tagliaferri, "OrientDB: A NoSQL, open source MMDMS," in *Proceedings of the British International Conference on Databases (BICOD)*, 2021.

[8] C. Himpe, "DatAasee – a metadata-lake as metadata catalog for a virtual data-lake," *arXiv preprint arXiv:2409.05512*, 2024.

[9] N. Bajiya, I. Gupta, and G. P. S. Raghava, "AptBacterialDB: A comprehensive, manually curated database of antibacterial aptamers," bioRxiv preprint, 2026, doi:10.64898/2026.07.01.735956; uses ArcadeDB/openCypher for the aptamer–target knowledge graph.

[10] P. O'Neil, E. Cheng, D. Gawlick, and E. O'Neil, "The log-structured merge-tree (LSM-tree)," *Acta Informatica*, vol. 33, no. 4, pp. 351–385, 1996.

[11] D. Ongaro and J. Ousterhout, "In search of an understandable consensus algorithm," in *USENIX Annual Technical Conference*, 2014, pp. 305–319.

[12] C. Chen, C. Jin, Y. Zhang, S. Podolsky, C. Wu, S.-P. Wang, E. Hanson, Z. Sun, R. Walzer, and J. Wang, "SingleStore-V: An integrated vector database system in SingleStore," *Proceedings of the VLDB Endowment*, vol. 17, no. 12, pp. 3772–3785, 2024.

[13] N. Upreti, H. V. Simhadri, H. S. Sundar, K. Sundaram *et al.*, "Cost-effective, low latency vector search with Azure Cosmos DB," *Proceedings of the VLDB Endowment*, vol. 18, no. 12, pp. 5166–5183, 2025.

[14] JVector, "JVector: A pure Java embedded vector search engine," https://github.com/datastax/jvector, 2024.

[15] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2020.

[16] S. J. Subramanya, Devvrit, R. Kadekodi, R. Krishnaswamy, and H. V. Simhadri, "DiskANN: Fast accurate billion-point nearest neighbor search on a single node," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[17] T. Formal, B. Piwowarski, and S. Clinchant, "SPLADE: Sparse lexical and expansion model for first stage ranking," in *Proceedings of the International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2021, pp. 2288–2292.

[18] A. Z. Broder, D. Carmel, M. Herscovici, A. Soffer, and J. Zien, "Efficient query evaluation using a two-level retrieval process," in *Proceedings of the ACM International Conference on Information and Knowledge Management (CIKM)*, 2003, pp. 426–434.

[19] S. Ding and T. Suel, "Faster top-k document retrieval using block-max indexes," in *Proceedings of the International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2011, pp. 993–1002.

[20] N. Tonellotto, C. Macdonald, and I. Ounis, "Efficient query processing for scalable web search," *Foundations and Trends in Information Retrieval*, vol. 12, no. 4–5, pp. 319–500, 2018.

[21] K. P. Gaffney, M. Prammer, L. Brasfield, D. R. Hipp, D. Kennedy, and J. M. Patel, "SQLite: Past, present, and future," *Proceedings of the VLDB Endowment*, vol. 15, no. 12, pp. 3535–3547, 2022.

[22] M. Raasveldt and H. Mühleisen, "Data management for data science — towards embedded analytics," in *Proceedings of the 10th Conference on Innovative Data Systems Research (CIDR)*, 2020. [Online]. Available: https://www.cidrdb.org/cidr2020/papers/p23-raasveldt-cidr20.pdf

[23] J. Wang, X. Yi, R. Guo, H. Jin, P. Xu, S. Li, X. Wang, X. Guo *et al.*, "Milvus: A purpose-built vector data management system," in *Proceedings of the ACM SIGMOD International Conference on Management of Data*, 2021, pp. 2614–2627.

[24] R. Guo, X. Luan, L. Xiang, X. Yan, X. Yi, J. Luo, Q. Cheng, W. Xu *et al.*, "Manu: A cloud native vector database management system," *Proceedings of the VLDB Endowment*, vol. 15, no. 12, pp. 3548–3561, 2022.

[25] C. Wei, B. Wu, S. Wang, R. Lou, C. Zhan, F. Li, and Y. Cai, "AnalyticDB-V: A hybrid analytical engine towards query fusion for structured and unstructured data," *Proceedings of the VLDB Endowment*, vol. 13, no. 12, pp. 3152–3165, 2020.

[26] W. Yang, T. Li, G. Fang, and H. Wei, "PASE: Postgresql ultra-high-dimensional approximate nearest neighbor search extension," in *Proceedings of the ACM SIGMOD International Conference on Management of Data*, 2020, pp. 2241–2253.

[27] Q. Zhang, S. Xu, Q. Chen, G. Sui, J. Xie, Z. Cai, Y. Chen, Y. He, Y. Yang, F. Yang, M. Yang, and L. Zhou, "VBASE: Unifying online vector similarity search and relational queries via relaxed monotonicity," in *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2023, pp. 377–395.

[28] S. Bruch, F. M. Nardini, C. Rulli, and R. Venturini, "Efficient inverted indexes for approximate retrieval over learned sparse representations," in *Proceedings of the International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2024.

[29] M. Raasveldt and H. Mühleisen, "DuckDB: An embeddable analytical database," in *Proceedings of the 2019 ACM SIGMOD International Conference on Management of Data*, 2019, pp. 1981–1984.

[30] M. A. Olson, K. Bostic, and M. Seltzer, "Berkeley DB," in *USENIX Annual Technical Conference, FREENIX Track*, 1999, pp. 183–191.

[31] B. Chattopadhyay, P. Dutta, W. Liu, O. Tinn, A. McCormick, A. Mokashi, P. Harvey, H. Gonzalez *et al.*, "Procella: Unifying serving and analytical data at YouTube," *Proceedings of the VLDB Endowment*, vol. 12, no. 12, pp. 2022–2034, 2019.

[32] B. Dageville, T. Cruanes, M. Zukowski, V. Antonov, A. Avanes, J. Bock, J. Claybaugh, D. Engovatov *et al.*, "The snowflake elastic data warehouse," in *Proceedings of the ACM SIGMOD International Conference on Management of Data*, 2016, pp. 215–226.